Attestation-Based Storage Provisioning for Linux



Nandakumar Raghavan <naraghavan@linux.microsoft.com>

Linux Systems Group, Microsoft

Linux Plumbers Conference 2026

Problem: The Mechanism Exists, the Workflow Does Not

Today, three integration areas are assembled with platform-specific glue.

Provisioning handoff

- repart creates and encrypts volume
- Post-step discovers enrolled keyslot
- cryptsetup imports provider token
- Deployment handles partial completion

Early-boot assembly

- systemd-cryptsetup + libcryptsetup
- Plugin, libraries and evidence support
- Trust material, network and DNS
- Verifier / KBS communication

Failure and recovery

- Rejection differs from outage
- Missing handler differs from bad metadata
- Wrong secret differs from no decision
- Retry / fail / recovery is deployment-specific

The missing piece is the contract across provisioning, boot dependencies, result propagation, and recovery.

Proposal: A Common Linux Storage Workflow

Provider-neutral above the plugin boundary; provider-specific below it.

PROVISIONING

systemd-repart
Create encrypted volume

→

Keyslot enrollment
Receive actual slot number

→

LUKS2 token
Register metadata against slot

Plugin boundary: storage lifecycle above; evidence, verifier/KBS protocol and authorization below

EARLY BOOT

systemd-cryptsetup
Request selected token

→

libcryptsetup
Dispatch handler

→

Provider plugin
Attest + retrieve secret

→

LUKS2
Activate keyslot

Three Gaps Blocking the Common Workflow

Each gap belongs to a different lifecycle stage, and this is scoped to providers beyond TPM2, FIDO2 and PKCS11 which already work end to end today.

Gap 1 | Provisioning

- Register custom token metadata with keyslot enrollment.
- Bind to the actual keyslot at creation; a crashed run must recreate the partition.
- PR: EncryptAddToken= / PR #42089 (merged).

Gap 2 | Token Failure Classification

- Classify token activation failuresin
- Distinguish rejection, outage, missing handler, bad metadata and wrong secret.

Gap 3 | initrd integration

- Package plugin, dependencies, trust, evidence support, network and diagnostics.
- Define bounded behavior and explicit recovery.

Gap 1: Provisioning

Merge token provisioning with the keyslot lifecycle.

Current risk

- A post-step script must rediscover the device and enrolled keyslot.
- Split durable updates create partial-completion and repair problems.

Required invariant

- Token metadata must reference the keyslot holding the matching secret.
- Never assume keyslot 0 and never store the unlock secret in token JSON.

Safe ordering

- Enroll secret → obtain actual keyslot → construct metadata → register token.
- Preserves existing tokens; skips (does not repair) an already-existing partition.

PR

- Issue #41565 and merged PR #42089 add EncryptAddToken=: option to register custom LUKS2 token during provisioning time

Gap 2: Classify Token Failures

Behavior varies	• Explicit token selection and CRYPT_ANY_TOKEN can follow different paths. • Caller behavior depends on the propagated result and configured alternatives.
Test matrix	• Success, policy denial, outage, missing handler, malformed metadata, and wrong secret. • Observe plugin → libcryptsetup → systemd → user-visible diagnostic.
Decision rule	• Use existing token APIs if policy-relevant distinctions survive. • Change API/ABI only if a required distinction is demonstrably lost.
Author's prototype	• Proposed: an opt-in token-policy option in systemd-cryptsetup (EAGAIN retry / EPERM deny); default unchanged unless set. Not yet implemented.

Ownership boundaries

Storage-owned	• Token envelope, keyslot binding, registration, selection and activation. • Generic tooling does not interpret provider protocols.
Provider-owned	• Evidence collection, verifier/KBS authentication and secret retrieval. • Provider-private metadata remains opaque and versioned.
Distribution-owned	• Initrd packaging, trust material, device access, network, diagnostics and recovery.
Trust rule	• Disk metadata may identify a resource; trusted policy authorizes it. • The unlock secret is returned through the authenticated plugin path, never stored in JSON.

Gap 3: Initrd Integration

A plugin that works after boot may fail for root-volume activation.

Required contents

- Plugin, libraries, evidence helper/device access, trust roots and configuration.

Connectivity

- Network and DNS must work before unlock.
- network-online does not prove verifier/KBS reachability.

Bounded behavior

- Provider requests must terminate predictably for unattended boot.
- Do not assume systemd can cancel an arbitrary blocking plugin call.

Recovery

- Provide actionable diagnostics and explicit recovery.
- Required-attestation policy must not silently degrade into bypass.

Security Claims and Non-Claims

Claim	• A verifier/KBS can condition release on evidence and policy. • Token binding connects the returned secret to a specific keyslot.
Non-claims	• Attestation alone does not protect plaintext after unlock. • It does not automatically provide rollback protection or runtime integrity.
Assumptions	• Fresh evidence, authenticated verifier/KBS, protected trust roots and recipient/session binding. • A suitable execution boundary protects the delivered secret.
Scope	• Storage integration, activation semantics and early-boot contract. • Not a new attestation protocol or universal broker implementation.

Thank you